Database Sharding, Partition Keys & Consistent Hashing

How distributed databases partition billions of records across shard clusters using consistent hash rings with virtual nodes.

1. Why Single Databases Fail at Scale

A single PostgreSQL instance caps out at ~50,000 QPS due to lock contention and connection pooling limits. Sharding splits tables horizontally across independent database servers.

```
Users 1 to 50M ➔ Shard A | Users 51M to 100M ➔ Shard B
```

2. Modulo Hashing vs Consistent Hashing

Modulo hashing `hash(key) % N` fails when adding a server: 100% of data must be moved. Consistent Hashing places nodes and keys on a $2^{32}-1$ circular ring:

```
Adding Node N+1: ONLY 1/(N+1) keys migrate! 90% data stays intact!
```

3. Virtual Nodes Prevent Hotspots

In small clusters, random node placement causes uneven distribution. Solution: Assign each physical machine 256 Virtual Nodes (v-nodes) spread evenly around the hash ring.

- Zero celebrity hotspot bottlenecks
- Uniform CPU & RAM balance across all shards

4. Shard Routing Architecture

How clients query sharded databases without know shard topology:

```
Client ➔ Envoy / Vitess Proxy ➔ Hash(user_id) ➔ Direct Shard Query
```

- Avoid cross-shard joins via Denormalization
- Use distributed transaction coordinators (2PC / Sagas)